

Công nghệ phần mềm

Thiết kế kiến trúc

Nội dung

- Định nghĩa KTPM & Sự quan trọng của KTPM
- Cấu trúc thiết kế
- Thuộc tính chất lượng
- Các nguyên lý thiết kế
- Các bước thiết kế kiến trúc
- Một số kiểu kiến trúc

Định nghĩa KTPM

- Có nhiều trường phái
- Len Bass và đồng nghiệp

*“The software architecture of a system is the set of **structures** needed to reason about the system, which comprise **software elements**, **relations** among them, and **properties** of both”*

- Chúng ta sẽ sử dụng định nghĩa của Len Bass và đồng nghiệp

Sự quan trọng của KTPM

- Hỗ trợ cho giao tiếp giữa các bên liên quan (stakeholders)
- Xác định các ràng buộc cho việc hiện thực hóa
 - Kiến trúc phần mềm là bản thiết kế mức cao của hệ thống
 - Những thiết kế chi tiết về sau sẽ được thực hiện dựa trên những ràng buộc có trong kiến trúc phần mềm
- Dự đoán một số tính chất của hệ thống (an ninh, hiệu năng,...)
- Nâng cao độ chính xác của việc dự đoán chi phí và thời gian xây dựng hệ thống

*“The software architecture of a system is the set of **structures** ...”*

CẤU TRÚC THIẾT KẾ

Định nghĩa

- Cấu trúc thiết kế = elements + mối quan hệ giữa các elements
- Một hệ thống phần mềm được thể hiện từ nhiều cấu trúc thiết kế khác nhau
 - Mỗi loại cấu trúc thể hiện một khía cạnh khác nhau của hệ thống
- Khi một cấu trúc thiết kế được sử dụng phổ biến để giải quyết một vấn đề cụ thể, ta gọi đó là mẫu kiến trúc (architecture pattern)

3 Loại cấu trúc thiết kế

- Loại cấu trúc mô đun (module structures)
 - Thể hiện hệ thống là tập các phần mã nguồn (classes, layers,...)
 - Trả lời câu hỏi:
 - Các chức năng chính của từng mô đun là gì?
 - Mối quan hệ phụ thuộc giữa các mô đun.
 - Ý nghĩa: lý giải được khả năng chỉnh sửa hệ thống

3 Loại cấu trúc

- Loại cấu trúc thành phần – kết nối (component-and-connector structures)
 - Thể hiện các thành phần của hệ thống và sự tương tác giữa các thành phần trong thời gian thực thi (runtime)
 - Trả lời câu hỏi
 - Thành phần nào là thành phần chính và tương tác với các thành phần khác như thế nào?
 - Phần nào của hệ thống sẽ được nhân bản (duplicated)?
 - Dữ liệu được xử lý trong hệ thống như thế nào?
 - Những phần nào có thể chạy song song?
 - Ý nghĩa: lý giải về hiệu năng, tính bảo mật, tính hợp lệ

3 Loại cấu trúc

- Loại cấu trúc phân phối (allocation structures)
 - Thể hiện mối quan hệ giữa hệ thống và các phần khác (CPU, file systems, networks,...)
 - Trả lời các câu hỏi: Thành phần nào sẽ chạy trên bộ xử lý nào? Các mô đun sẽ được lưu trữ trên thư mục, file nào? Các mô đun được phân công cho đội phát triển như thế nào?

CÁC NGUYÊN LÝ THIẾT KẾ

Các nguyên lý thiết kế

- Phân tách các khía cạnh quan tâm (Separation of concerns)
 - chia ứng dụng thành các phần càng ít sự chồng chéo về chức năng càng tốt
 - Cố gắng hạn chế tối đa sự tương tác giữa các thành phần nhằm có giảm sự phụ thuộc và tăng cường sự kết dính (cohesion) trong từng thành phần
- Trách nhiệm đơn
 - Mỗi thành phần chỉ thực hiện một chức năng hoặc một tập các chức năng gắn kết chặt chẽ

Các nguyên lý thiết kế

- Hiểu biết tối thiểu
 - Các thành phần không cần biết chi tiết bên trong của các thành phần khác
- Không lặp lại
 - Mỗi một chức năng chỉ được hiện thực hóa bởi một thành phần
- Hạn chế thiết kế trước
 - Chỉ thiết kế khi cần và có đủ thông tin

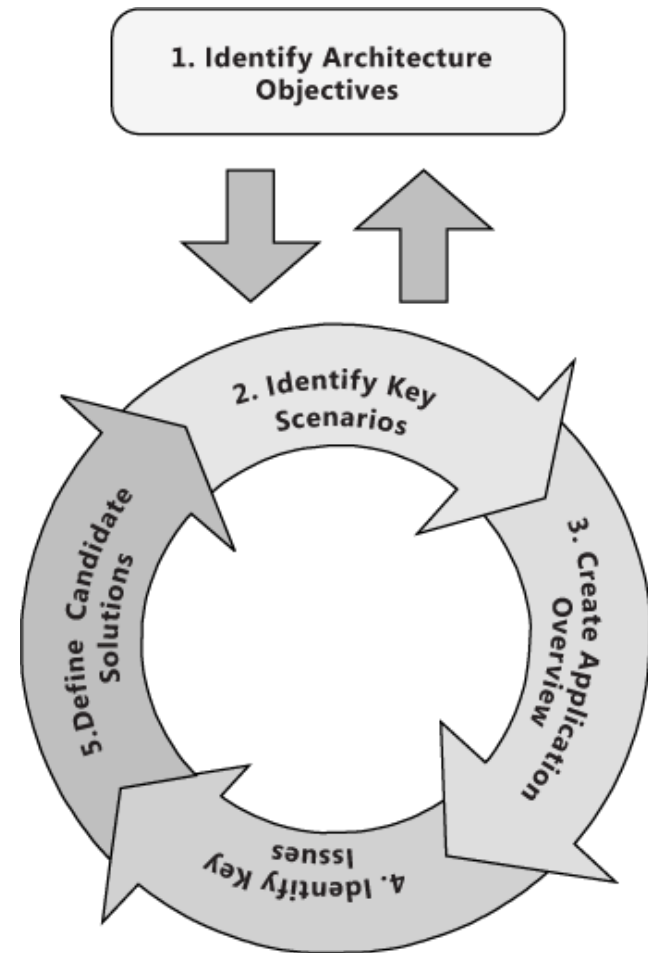
CÁC BƯỚC THIẾT KẾ KIẾN TRÚC

Thiết kế kiến trúc

- Đầu vào: yêu cầu chức năng, phi chức năng (các ràng buộc), v.v.
- Đầu ra: bản thiết kế kiến trúc
 - Thuộc một loại cấu trúc kiến trúc nào đó
- Là quá trình lặp với 5 bước chính

Các bước chính

1. Xác định mục tiêu
2. Xác định các hoạt cảnh sử dụng chính
3. Xác định tổng quan về ứng dụng
4. Xác định các vấn đề chính
5. Xác định các giải pháp chính



B1. Xác định mục tiêu

- Thiết kế kiến trúc để làm gì
- Cho ai?
- Các ràng buộc là gì?



Phạm vi và thời gian thực hiện

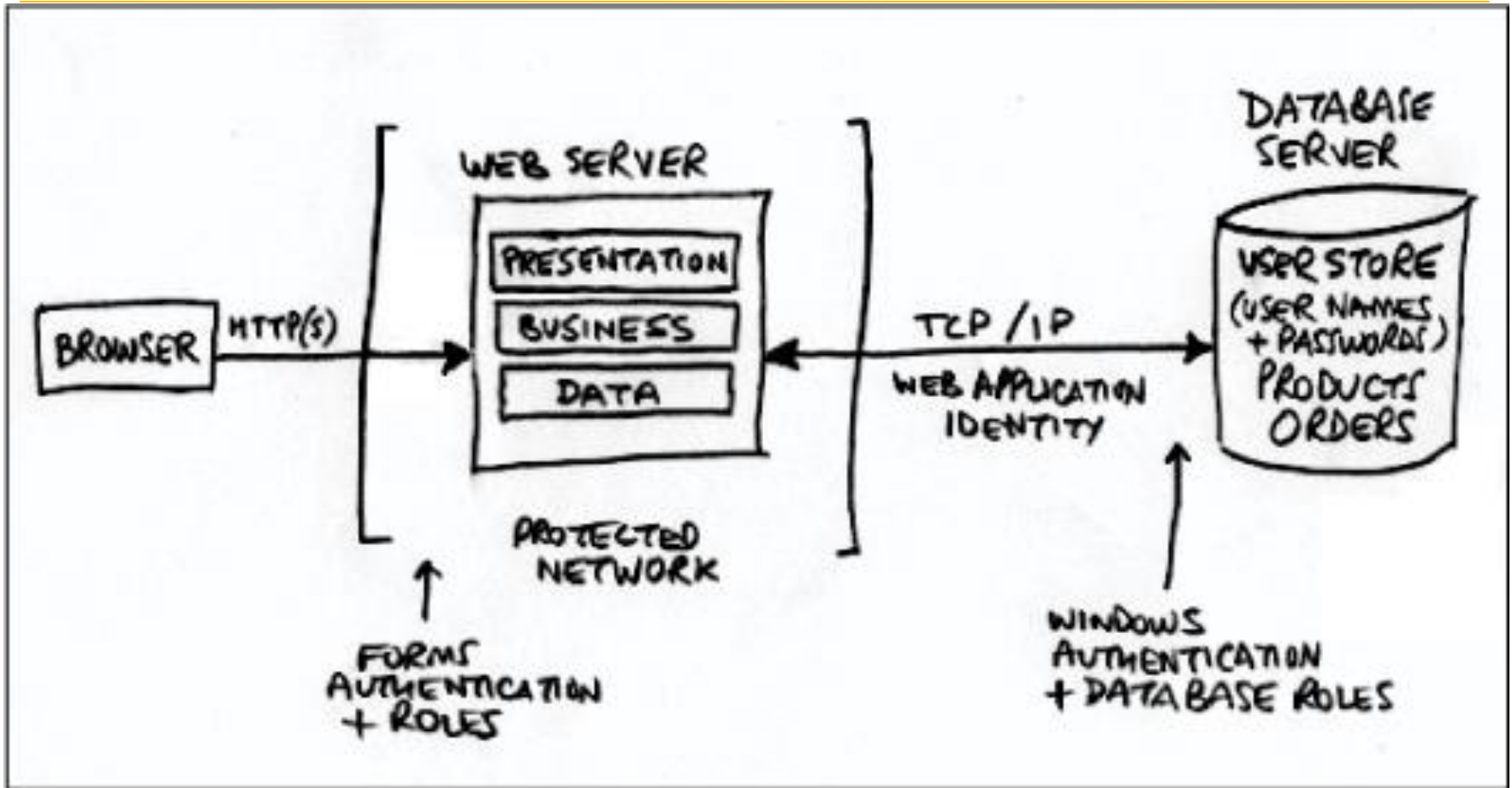
B2. Hoạt cảnh sử dụng chính

- Một hoạt cảnh là sự tổng quát của nhiều ca sử dụng (use case) tương tự
- Các hoạt cảnh chính
 - Có ảnh hưởng lớn, được sử dụng nhiều
 - Thể hiện sự “đánh đổi” giữa các thuộc tính chất lượng

B3. Tổng quan về ứng dụng

- Xác định kiểu ứng dụng
- Xác định các ràng buộc triển khai
- Các kiểu kiến trúc có thể sử dụng
- Các công nghệ liên quan

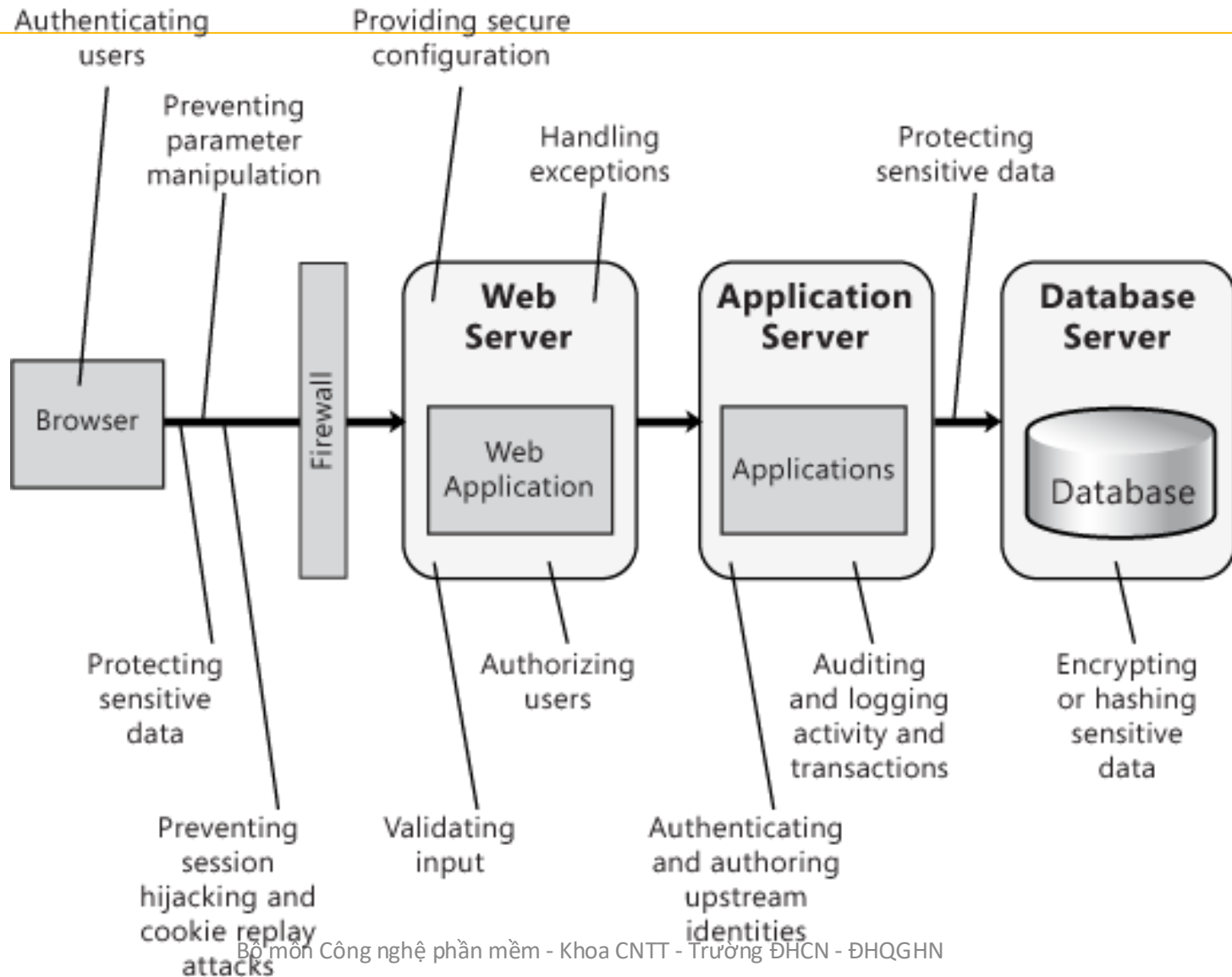
Phác thảo kiến trúc



B4. Xác định các vấn đề chính

- Đánh giá các thuộc tính chất lượng
 - Tính hợp lệ
 - Tính linh hoạt
 - Tính khả chuyển
 - Tính sử dụng lại
 - Tính hiệu năng
 - Tính an ninh

Xác định các vấn đề chính



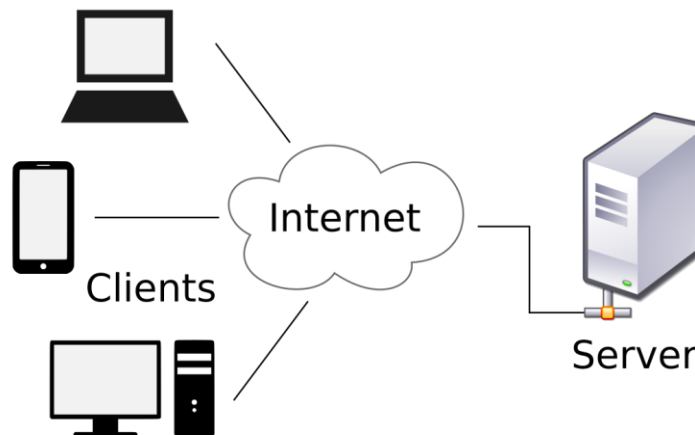
B5. Giải pháp dự kiến

- Là bản thiết kế kiến trúc ở mức cao
- Bao gồm cả kiểu ứng dụng, kiến trúc triển khai, kiểu kiến trúc, công nghệ được lựa chọn, thuộc tính chất lượng, các khía cạnh khác
- Kiểm tra các yêu cầu, ràng buộc

MỘT SỐ KIỂU KIẾN TRÚC

Kiểu Khách/Chủ

- Cho hệ thống phân tán với hai thành phần riêng biệt Khách (client) và Chủ (server)
- Thuộc cấu trúc thành phần – kết nối
- Cấu hình 1-1, 1-n, n-1
- Trước đây, Client thường là ứng dụng chạy trên desktop; hiện nay có thêm Web browser và ứng dụng di động



Kiểu Khách/Chủ

- Một số biến thể
 - Client-Queue-Client: cho phép các clients có thể giao tiếp thông qua hàng đợi. Server chỉ đóng vai trò là hàng đợi để chứa dữ liệu. Còn được gọi là kiến trúc hàng đợi bị động
 - Peer-to-Peer: Client và server có thể đổi vai trò cho nhau

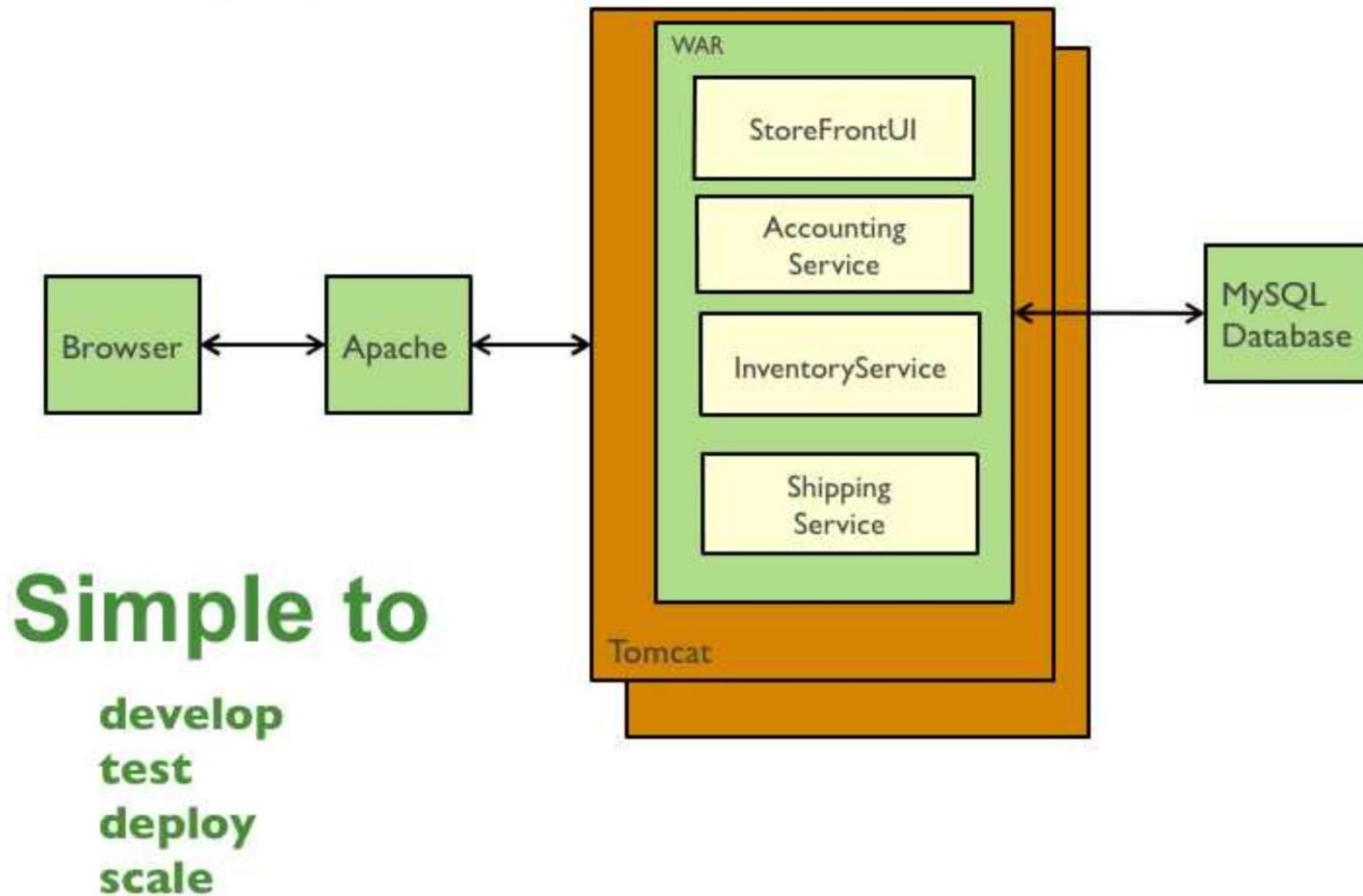
Kiến trúc Monolithic

- Giao diện người dùng và mã nguồn logic được kết hợp thành 1 chương trình duy nhất từ một nền tảng duy nhất
- Ví dụ
 - Ban đầu, Netflix, Amazon, và eBay dùng kiến trúc Monolithic
 - Hầu hết các ứng dụng web theo kiến trúc này

Kiến trúc Monolithic

- Ví dụ: Bạn đang xây dựng 1 trang web bán giày
 - Ứng dụng có các thành phần như StoreFrontUI (cài đặt giao diện), backend (kiểm tra credit khách hàng, tạo order, quản lý kho, v.v.)
 - Công nghệ sử dụng là JavaEE
 - Thiết kế ứng dụng theo kiến trúc Monolithic
 - Sản phẩm được đóng gói thành 1 tệp WAR duy nhất
 - Tệp WAR này được chạy trên một web container như Tomcat

Kiến trúc Monolithic



Kiến trúc Monolithic

- Ưu điểm
 - Dễ phát triển: nhiều IDE hỗ trợ phát triển các ứng dụng dạng này
 - Dễ triển khai: chỉ cần tạo ra file WAR và đưa lên server để chạy
 - Dễ mở rộng: Có thể mở rộng ứng dụng bằng cách tạo nhiều bản sao của file WAR

Kiến trúc Monolithic

- Nhược điểm khi mã nguồn đủ lớn
 - Không có sự phân tách rõ ràng giữa các thành phần trong mã nguồn nên khá khó để hiểu
 - Web container tốn nhiều thời gian để khởi chạy tệp WAR
 - Tạo nhiều bản sao của WAR có thể làm chậm server do tốn RAM hơn và vấn đề I/O

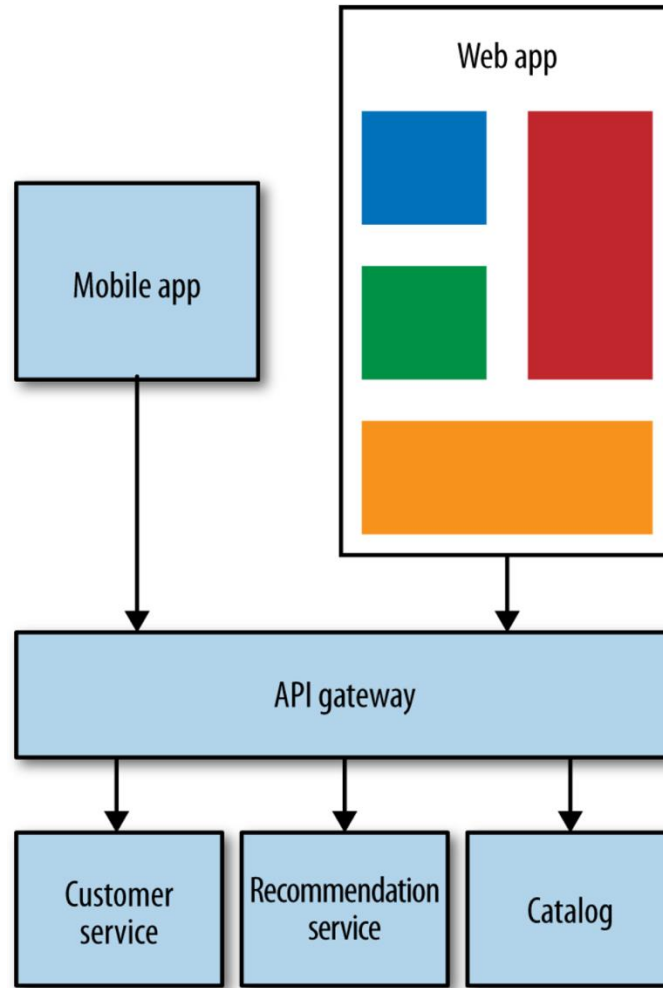
Kiến trúc Monolithic

- Nhược điểm khi mã nguồn đủ lớn
 - Không phù hợp khi có nhiều đội phát triển (mỗi đội đảm nhận một vai trò riêng)
 - Khi cập nhật một module dù nhỏ, phải build lại WAR từ đầu và đẩy lên server -> phiền toái
 - Khó áp dụng công nghệ mới vào

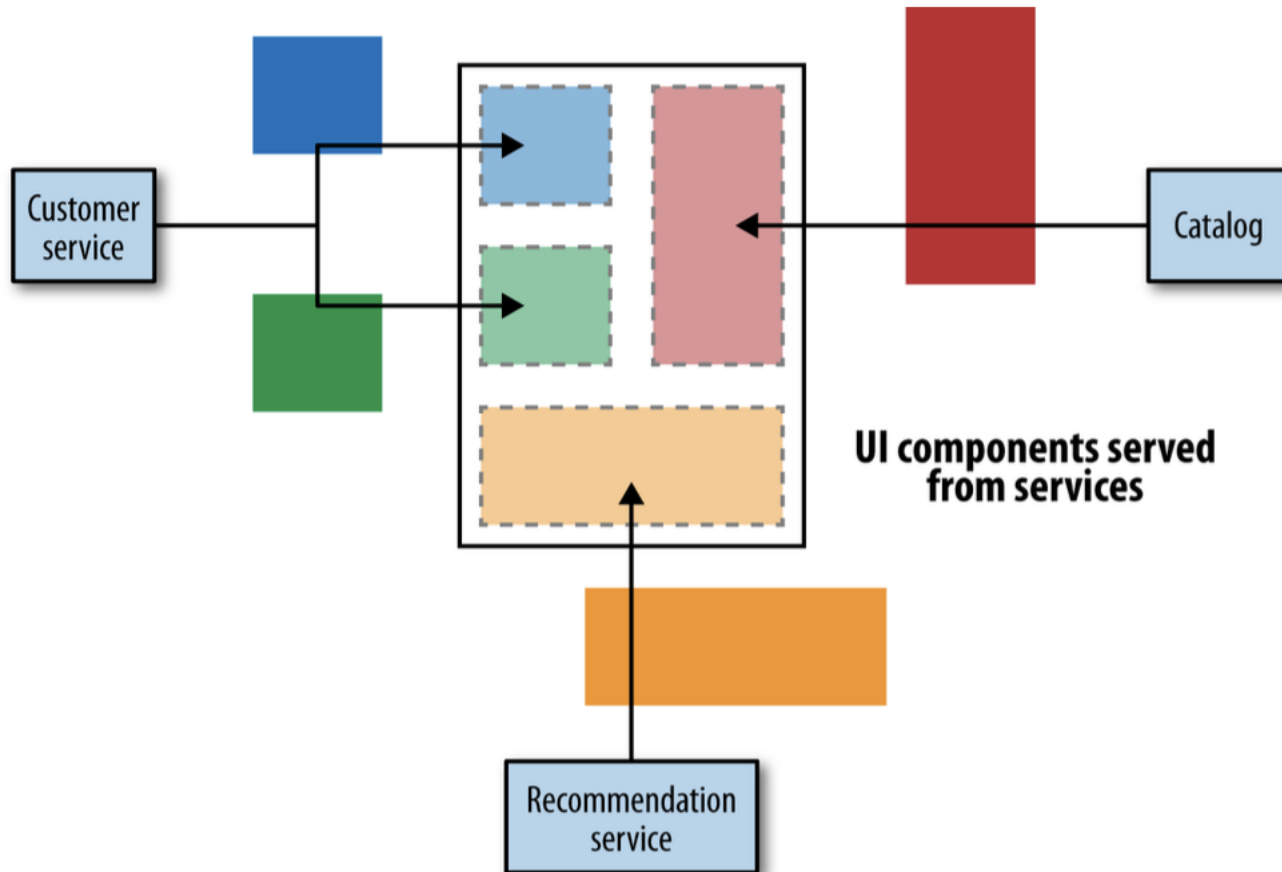
Kiến trúc Microservice

- Giải quyết các vấn đề của kiến trúc Monolithic
- Kiến trúc gồm một tập các service
 - Mỗi service có một vai trò riêng, có thể được phát triển và triển khai độc lập với các service khác
 - Các service thường giao tiếp với nhau qua HTTP/REST
- Các công ty hiện tại dùng kiến trúc Microservice có thể kể đến Comcast Cable, Uber, Netflix, Amazon, Ebay, Sound Cloud, Capital One, v.v.

Kiến trúc Microservice

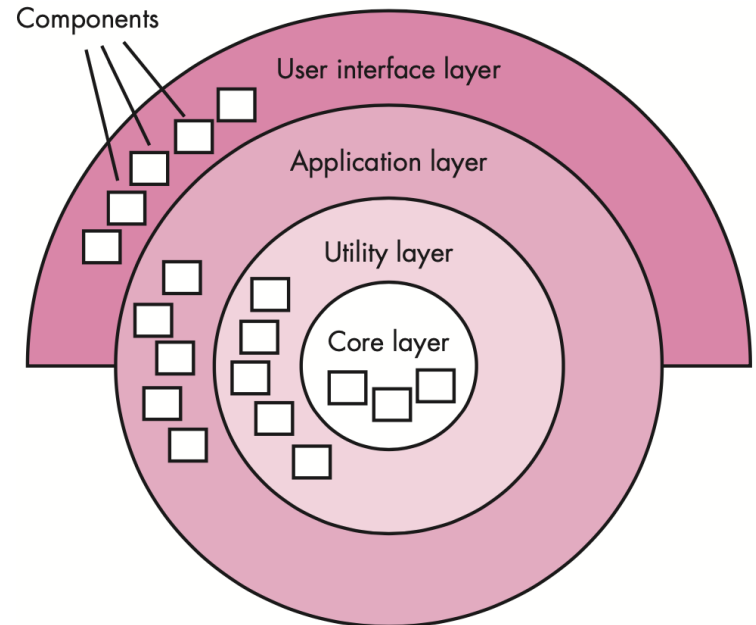


Kiến trúc Microservice



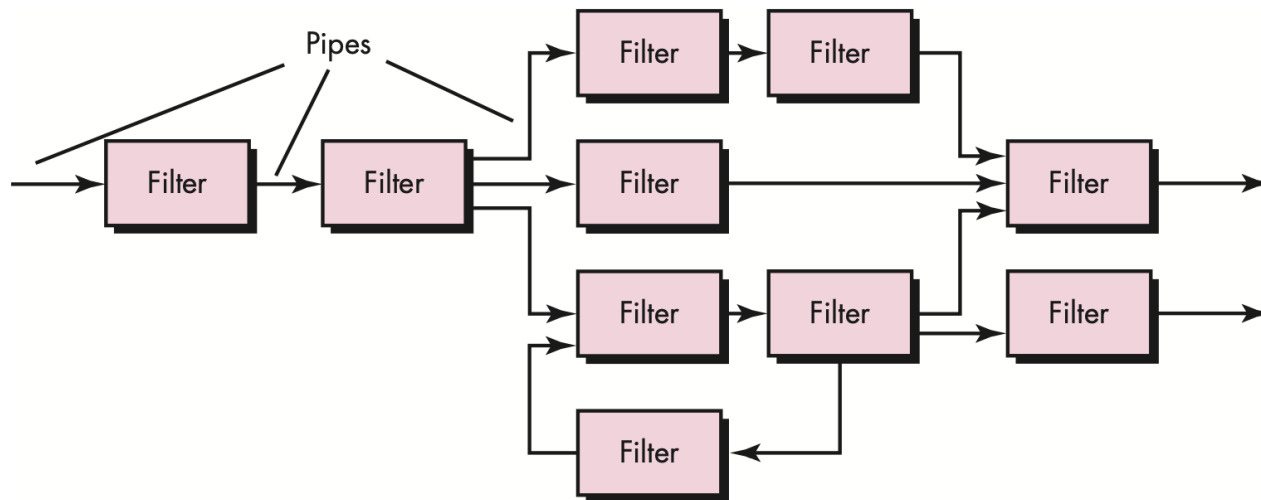
Kiểu phân lớp

- Nhóm các chức năng có liên quan chặt chẽ vào chung một lớp
- Các lớp được sắp xếp chồng lên nhau
- Chỉ lớp trên mới sử dụng chức năng của lớp dưới



Kiểu ống dẫn và bộ lọc

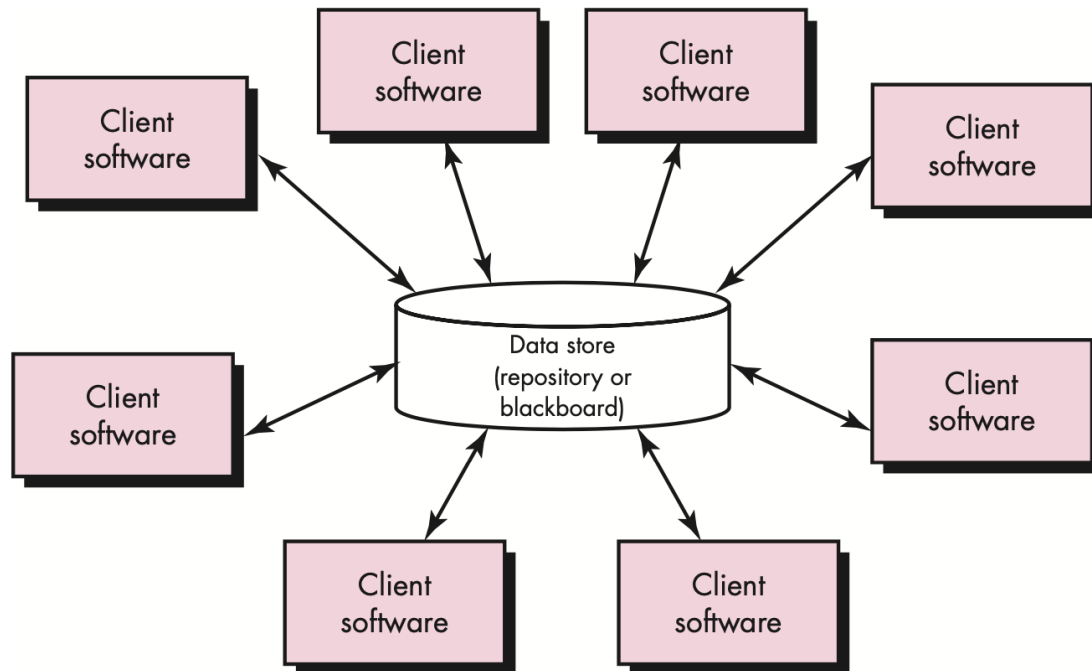
- Bộ lọc (filter): nơi xử lý dữ liệu
 - Các bộ lọc nên hoạt động độc lập với các bộ lọc khác
- Ống dẫn (pipe): đường truyền dữ liệu



Pipes and filters

Kiến trúc hướng dữ liệu

- Data store: nơi lưu trữ dữ liệu dùng chung
- Client software: truy cập đến dữ liệu, có thể thêm/xóa/sửa



THUỘC TÍNH CHẤT LƯỢNG CỦA KIẾN TRÚC PHẦN MỀM

Định nghĩa “thuộc tính chất lượng”

- Chúng ta cần có các tiêu chí để đánh giá chất lượng của KTPM, gọi đó là các thuộc tính chất lượng
- Một thuộc tính chất lượng là một độ đo hệ thống có thể đánh giá được và định lượng được, từ đó làm rõ mức độ mà hệ thống thỏa mãn yêu cầu của các bên liên quan

Các thuộc tính chất lượng

- Tính hợp lệ (Availability): đo mức độ sẵn sàng của hệ thống để thực hiện tác vụ khi cần
 - Công thức: MTBF (thời gian trung bình giữa các lần xảy ra lỗi), MTTR (thời gian trung bình để sửa lỗi)

$$\frac{MTBF}{(MTBF + MTTR)}$$

Availability	Downtime/90 Days	Downtime/Year
99.0%	21 hours, 36 minutes	3 days, 15.6 hours
99.9%	2 hours, 10 minutes	8 hours, 0 minutes, 46 seconds
99.99%	12 minutes, 58 seconds	52 minutes, 34 seconds
99.999%	1 minute, 18 seconds	5 minutes, 15 seconds
99.9999%	8 seconds	32 seconds

Các thuộc tính chất lượng

- Tính linh hoạt: Chúng ta có thể thay đổi thành phần X không?
- Tính khả chuyển: Chúng ta có thể triển khai trên một máy khác không?
- Tính sử dụng lại: Chúng ta có thể sử dụng lại một phần hay toàn bộ cho ứng dụng khác không?

Các thuộc tính chất lượng

- Tính hiệu năng: thời gian hệ thống phản hồi tương tác của người dùng
- Tính an ninh: khả năng hệ thống bảo vệ dữ liệu và thông tin từ các truy cập bất hợp pháp, trong khi đó vẫn cho phép người dung/hệ thống hợp lệ được truy cập

Hướng dẫn cho một vài thuộc tính

- Cải thiện thuộc tính hiệu năng
 - Phân rã thành các tiến trình chạy song song
 - Quản lý lượng và tần suất dữ liệu truyền nhận giữa các tiến trình
 - Xác định “nút cổ chai” về hiệu năng trong hệ thống
- Cải thiện thuộc tính an ninh
 - Phân chia chức năng cần và không cần login
 - Cấu trúc phần mềm thành nhiều tầng

Hỏi - Đáp

Tài liệu tham khảo

- Len Bass et al., Software architecture in practice, 3rd Edition
- Microsoft, Microsoft Application Architecture Guide, 2nd Edition